



ASIC Verification
EE5213
Exercise 2 Report
FSM Reachability Analysis

Student: Pham Huy Thanh - 2570317

Instructor: TS. Tran Hoang Linh

April 1, 2026

1 Exercise 2: FSM Reachability Analysis

1.1 Objective

Implement the State Reachability Analysis algorithm for an arbitrary Finite State Machine (FSM) using C++. The program computes the set of reachable states given a set of initial states and a transition relation.

1.2 Design Specifications

- **Input:** Set of initial states (I), and a transition relation (R) mapping current states to next possible states.
- **Output:** The complete set of reachable states (ARS - All Reached States).
- **Algorithm Strategy:**
 - Track **All Reached States** (ARS) and **New Next States** (NNS or Frontier Set).
 - Iteratively compute the next states (NS) from the current frontier NNS .
 - Update the frontier as newly discovered states ($NNS = NS - ARS$).
 - Add the new frontier to the all reached states ($ARS = ARS \cup NNS$).
 - Terminate when the frontier is empty.

1.3 Example FSM Analysis

To illustrate the algorithm, consider the FSM defined in the implementation with the following transition relations:

- $S_0 \rightarrow \{S_1, S_5\}$
- $S_1 \rightarrow \{S_2\}$
- $S_5 \rightarrow \{S_5, S_2\}$
- $S_2 \rightarrow \{S_3\}$
- $S_3 \rightarrow \{S_4\}$
- $S_4 \rightarrow \emptyset$

The initial state set is $I = \{S_0\}$. The state diagram is shown in Figure 1.

Step-by-Step Execution:

1. Initialization:

- $ARS = \{S_0\}$
- Frontier $NNS = \{S_0\}$

2. Iteration 1:

- Compute next states from NNS : $NS = \{S_1, S_5\}$

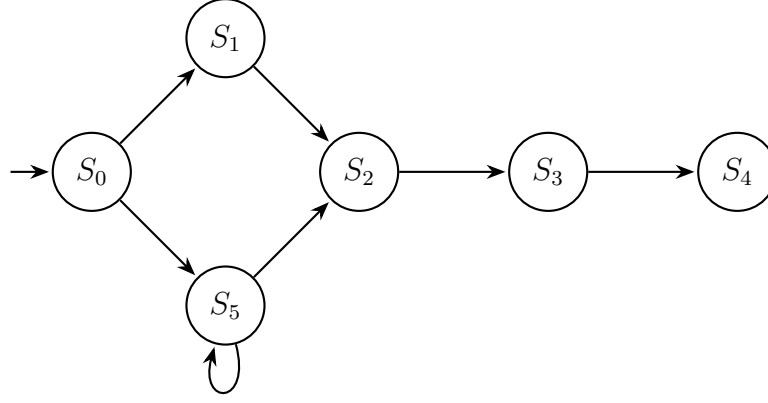


Figure 1: State Diagram of the Example FSM

- Compute new frontier: $NNS = NS \setminus ARS = \{S_1, S_5\} \setminus \{S_0\} = \{S_1, S_5\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_5\}$

3. Iteration 2:

- Compute next states from NNS : $NS = \{S_2, S_5\}$ (from $S_1 \rightarrow S_2$ and $S_5 \rightarrow S_2, S_5$)
- Compute new frontier: $NNS = NS \setminus ARS = \{S_2, S_5\} \setminus \{S_0, S_1, S_5\} = \{S_2\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_5\}$

4. Iteration 3:

- Compute next states from NNS : $NS = \{S_3\}$ (from $S_2 \rightarrow S_3$)
- Compute new frontier: $NNS = NS \setminus ARS = \{S_3\} \setminus \{S_0, S_1, S_2, S_5\} = \{S_3\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_3, S_5\}$

5. Iteration 4:

- Compute next states from NNS : $NS = \{S_4\}$ (from $S_3 \rightarrow S_4$)
- Compute new frontier: $NNS = NS \setminus ARS = \{S_4\} \setminus \{S_0, S_1, S_2, S_3, S_5\} = \{S_4\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_3, S_4, S_5\}$

6. Iteration 5:

- Compute next states from NNS : $NS = \emptyset$ (from $S_4 \rightarrow \emptyset$)
- Compute new frontier: $NNS = NS \setminus ARS = \emptyset \setminus ARS = \emptyset$
- The frontier NNS is empty, so the algorithm terminates.

Final Result: $ARS = \{S_0, S_1, S_2, S_3, S_4, S_5\}$

1.4 Implementation (fsm_reachability.cpp)

```
1 #include <iostream>
2 #include <vector>
3 #include <set>
4 #include <map>
5 #include <algorithm>
6
7 typedef int State;
8
9 std::set<State> Reachability(
10     const std::map<State, std::set<State>>& R,
11     const std::set<State>& I
12 ) {
13     std::set<State> ARS = I;    // All Reached States
14     std::set<State> NNS = I;    // New Next States (Frontier Set)
15
16     while (true) {
17         std::set<State> NS;
18         for (State s : NNS) {
19             if (R.find(s) != R.end()) {
20                 for (State next_s : R.at(s)) {
21                     NS.insert(next_s);
22                 }
23             }
24         }
25
26         std::set<State> new_NNS;
27         std::set_difference(
28             NS.begin(), NS.end(),
29             ARS.begin(), ARS.end(),
30             std::inserter(new_NNS, new_NNS.begin())
31         );
32         NNS = new_NNS;
33
34         if (NNS.empty()) {
35             break;
36         }
37         ARS.insert(NNS.begin(), NNS.end());
38     }
39     return ARS;
40 }
41
42 int main() {
43     std::map<State, std::set<State>> transition_relation = {
44         {0, {1, 5}}, {1, {2}}, {5, {5, 2}},
45         {2, {3}}, {3, {4}}, {4, {}}
46     };
47     std::set<State> initial_states = {0};
48     std::set<State> reachable_states = Reachability(transition_relation, initial_states);
49
50     std::cout << "Final Reachable States (ARS): { ";
51     for (State s : reachable_states) std::cout << "S" << s << " ";
52     std::cout << "}\n";
53     return 0;
54 }
```

1.5 Execution Results

```
1 Starting Reachability Analysis...
2 Initial States: { S0 }
3
4 Final Reachable States (ARS): { S0 S1 S2 S3 S4 S5 }
```